# SWE 261P Software Testing and Analysis - Part 6 Report

## PDFsam Basic: Static Analyzers

JAVA 21

CODEQL

<svg xmlns="http://www.w3.org/2000/svg" width="147.25" height="28" role="img" aria-label="SP(

MAVEN

Course SWE 261P
Quarter Winter 2026
Team Kingson & Zian & Zhenyu

**Repo Github Link:**
https://github.com/eric-song-dev/pdfsam

**CodeQL Security Alert Dashboard:**
https://github.com/eric-song-dev/pdfsam/security/code-scanning

**Team Members:**


- Kingson Zhang: kxzhang@uci.edu
- Zian Xu: zianx11@uci.edu
- Zhenyu Song: zhenyus4@uci.edu


This report documents the application of **static analysis** tools to **PDFsam Basic**, covering CodeQL (GitHub's code scanning tool), SpotBugs (bytecode-level bug finder), and a comparison of their findings.

# 📂 Quick Navigation


- SWE 261P Software Testing and Analysis - Part 6 Report
  - PDFsam Basic: Static Analyzers
  - 📂 Quick Navigation
  - 🎯 1. Static Analysis: Goals, Purposes, and Use
    - 1.1 What Is Static Analysis?
    - 1.2 Why Static Analysis Matters
    - 1.3 Static Analysis vs. Dynamic Testing
    - 1.4 Tools Used in This Report
  - 🔧 2. CodeQL Setup and Configuration
    - 2.1 What Is CodeQL?
    - 2.2 Enabling CodeQL via GitHub Actions
    - 2.3 Configuration Details
    - 2.4 CodeQL Findings Overview
  - 📊 3. SpotBugs Setup and Configuration
    - 3.1 What Is SpotBugs?
    - 3.2 Running SpotBugs
    - 3.3 SpotBugs Findings Overview
    - 3.4 Bug Categories Breakdown
  - 🔍 4. CodeQL Detailed Findings — Per Team Member
    - 4.1 Zhenyu's CodeQL Finding: Use of a Broken or Risky Cryptographic Algorithm
      - What the Warning Says
      - Is This an Actual Problem?
    - 4.2 Kingson's CodeQL Finding: Inefficient Regular Expression (ReDoS)
      - What the Warning Says
      - Is This an Actual Problem?
    - 4.3 Zian's CodeQL Finding
      - What the Warning Says
      - Is This an Actual Problem?
  - 🔍 5. SpotBugs Detailed Findings — Per Team Member
    - 5.1 Zhenyu's SpotBugs Finding: Inconsistent Synchronization
      - What the Warning Says
      - Is This an Actual Problem?
    - 5.2 Kingson's SpotBugs Finding
      - What the Warning Says
      - Is This an Actual Problem?
    - 5.3 Zian's SpotBugs Finding
      - What the Warning Says

- Is This an Actual Problem?
- ⚖️ 6. Tool Comparison
  - 6.1 Summary of Findings
  - 6.2 Overlap and Differences
  - 6.3 Overlapping Findings
  - 6.4 Strengths and Weaknesses
  - 6.5 Complementary Value
- 📋 7. Summary
  - 7.1 Files Changed
  - 7.2 Commands Used
  - 7.3 Key Takeaways
- 🎯 8. Conclusion

# 🎯 1. Static Analysis: Goals, Purposes, and Use

## 1.1 What Is Static Analysis?

**Static analysis** is a method of examining source code or compiled bytecode *without executing* the program. Static analyzers systematically search for patterns that indicate potential bugs, security vulnerabilities, code quality issues, or deviations from best practices. Unlike dynamic testing (which runs the code with specific inputs), static analysis reasons about *all possible* execution paths.

## 1.2 Why Static Analysis Matters

| Benefit | Description |
|---|---|
| **Early Bug Detection** | Identifies defects before code is even run — Microsoft reports that most bugs are found through static analysis |
| **Broad Coverage** | Analyzes all code paths, including rarely-executed error-handling and edge-case branches |
| **Security** | Detects vulnerabilities (injection, XSS, insecure crypto) that may escape functional testing |
| **Consistency** | Enforces coding standards and best practices uniformly across the entire codebase |
| **Cost Efficiency** | Finding bugs statically is far cheaper than finding them in production |
| **Automation** | Integrates into CI pipelines (e.g., GitHub Actions) for continuous quality monitoring |

## 1.3 Static Analysis vs. Dynamic Testing

| Aspect | Static Analysis | Dynamic Testing |
|---|---|---|
| **Execution** | No code execution | Runs the code |
| **Approach** | Pessimistic — flags anything *potentially* wrong | Optimistic — only finds bugs that trigger during test runs |
| **Coverage** | All code paths (including dead code) | Only paths exercised by test cases |
| **False Positives** | Can produce false alarms | Findings are actual failures (if test is correct) |

| Aspect | Static Analysis | Dynamic Testing |
|---|---|---|
| **False Negatives** | May miss runtime-specific issues | Misses any untested path |

## 1.4 Tools Used in This Report

| Tool | Type | Analysis Level | Integration |
|---|---|---|---|
| **CodeQL** | Semantic code analysis | Source code (AST + data flow) | GitHub Code Scanning |
| **SpotBugs** | Bytecode analysis | Compiled `.class` files | Maven plugin |

# 🔧 2. CodeQL Setup and Configuration

## 2.1 What Is CodeQL?

**CodeQL** is GitHub's semantic code analysis engine. Unlike pattern-matching linters, CodeQL builds a full relational database from the source code (including AST, control flow, and data flow) and then runs queries against it. This enables CodeQL to find deep, path-sensitive bugs such as:

- SQL injection and other injection vulnerabilities
- Null pointer dereferences across method boundaries
- Unvalidated user input flowing into security-sensitive operations
- Dead code, unused variables, and redundant conditions

## 2.2 Enabling CodeQL via GitHub Actions

We created a CodeQL workflow file to enable GitHub's Code Quality Scanning:

**Workflow File**: codeql.yml

```yaml
name: "CodeQL Analysis"

on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]

jobs:
  analyze:
    name: Analyze (Java)
    runs-on: ubuntu-latest
    permissions:
      security-events: write
      packages: read
      actions: read
      contents: read

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: maven

      - name: Initialize CodeQL
        uses: github/codeql-action/init@v4
        with:
          languages: java-kotlin
          queries: security-and-quality

      - name: Build with Maven
        run: mvn compile -DskipTests -Dmaven.antrun.skip=true --batch-mode

      - name: Perform CodeQL Analysis
        uses: github/codeql-action/analyze@v4
        with:
          category: "/language:java-kotlin"
```

## 2.3 Configuration Details

| Setting | Value | Purpose |
|---|---|---|
| **Language** | `java-kotlin` | Analyze all Java source files |
| **Query Suite** | `security-and-quality` | Includes both security vulnerability checks and code quality rules |
| **Trigger** | Push to `master` + PRs | Ensures all code is scanned before merging |
| **Build Step** | `mvn compile -DskipTests` | CodeQL needs compiled code to build its database |

## 2.4 CodeQL Findings Overview

**CodeQL Security Alert Dashboard:**
https://github.com/eric-song-dev/pdfsam/security/code-scanning

After enabling CodeQL on our repository, the analysis produced the following results (available in full at the Code Scanning Dashboard):

| Severity | Count | Description |
|---|---|---|
| **High** | 3 | Critical security vulnerabilities (Weak Crypto) or performance issues (ReDoS) |
| **Warning** | 15 | Code quality issues (e.g., Unsafe getResource, Useless null check) |
| **Note** | 66 | Informational suggestions (e.g., Useless parameter, Deprecated method) |
| **Total** | **84** | Total findings across all modules |

The findings primarily fall into these categories:

| Category | Count | Example Observation |
|---|---|---|
| **Useless Parameters** | 36 | Parameters that are never read/used inside methods |
| **Deprecated API Usage** | 16 | Invocation of deprecated methods or constructors |
| **Unsafe Resource Loading** | 14 | `Unsafe use of getResource` warnings |
| **Exception Handling** | 9 | `Missing catch of NumberFormatException` |
| **Code Style** | 3 | `Underscore used as identifier` |

| Category | Count | Example Observation |
| --- | --- | --- |
| **Security / Cryptography** | 2 | `java/weak-cryptographic-algorithm` — Weak Cipher algorithm usage |
| **Unused Variables** | 1 | `Unread local variable` |
| **Default String Forms** | 1 | `Use of default toString()` |
| **Performance (ReDoS)** | 1 | `java/redos` — Inefficient regular expression |
| **Redundant Logic** | 1 | `java/useless-null-check` — Useless null checks |

# 📊 3. SpotBugs Setup and Configuration

## 3.1 What Is SpotBugs?

**SpotBugs** is the successor to FindBugs, a well-known static analysis tool for Java. It analyzes compiled Java *bytecode* ( `.class` files) to find over 400 bug patterns grouped into categories like correctness, performance, security, multithreaded correctness, and malicious code vulnerability.

## 3.2 Running SpotBugs

We used the SpotBugs Maven plugin (version 4.8.6.6) to analyze the project's compiled bytecode:

```
# Run SpotBugs on all non-GUI modules
mvn compile com.github.spotbugs:spotbugs-maven-plugin:4.8.6.6:spotbugs -pl pdfsam-model,pdfsa

# View the XML report
cat pdfsam-model/target/spotbugsXml.xml
cat pdfsam-core/target/spotbugsXml.xml
cat pdfsam-persistence/target/spotbugsXml.xml
cat pdfsam-service/target/spotbugsXml.xml
```

## 3.3 SpotBugs Findings Overview

| Module | Bugs Found | Priority 1 (High) | Priority 2 (Medium) |
|---|---|---|---|
| `pdfsam-model` | 33 | 0 | 33 |
| `pdfsam-core` | 9 | 1 | 8 |
| `pdfsam-persistence` | 1 | 0 | 1 |
| `pdfsam-service` | 4 | 0 | 4 |
| **Total** | **47** | **1** | **46** |

## 3.4 Bug Categories Breakdown

| Category | Code | Count | Description |
|---|---|---|---|
| **Malicious Code (MALICIOUS_CODE)** | `EI_EXPOSE_REP` / `EI_EXPOSE_REP2` | 42 | Methods expose or store mutable |

| Category | Code | Count | Description |
|---|---|---|---|
| | | | internal object references |
| **Internationalization (I18N)** | `DM_DEFAULT_ENCODING` | 1 | Reliance on default platform encoding |
| **Multithreaded Correctness (MT_CORRECTNESS)** | `IS2_INCONSISTENT_SYNC` | 1 | Inconsistent synchronization of shared state |
| **Correctness (CORRECTNESS)** | `SING_SINGLETON_HAS_NONPRIVATE_CONSTRUCTOR` | 1 | Singleton class with accessible constructor |
| **Performance (PERFORMANCE)** | `EI_EXPOSE_REP` (subset) | 2 | Unnecessary defensive copies in hot paths |

# 🔍 4. CodeQL Detailed Findings — Per Team Member

## 4.1 Zhenyu's CodeQL Finding: Use of a Broken or Risky Cryptographic Algorithm

| Property | Value |
|---|---|
| **Rule** | `java/weak-cryptographic-algorithm` |
| **Severity** | High |
| **File** | EncryptionUtils.java |
| **Module** | `pdfsam-core` |

### What the Warning Says

CodeQL flags the use of an insecure cryptographic method ( `AES/ECB/PKCS5Padding` ) in `EncryptionUtils.decrypt()` :

```
// EncryptionUtils.java, Lines 73-77
public static String decrypt(String value) {
    try {
        if (nonNull(value)) {
            Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding"); // ← CodeQL: Risky al
            cipher.init(Cipher.DECRYPT_MODE,
                    new SecretKeySpec(Arrays.copyOf(T_KEY.getBytes(StandardCharsets.UTF_8), 1
            return new String(cipher.doFinal(Base64.getDecoder().decode(value)), StandardChar
```

According to CodeQL (CWE-327), cryptographic algorithm `AES/ECB/PKCS5Padding` is insecure because it uses the Electronic Codebook (ECB) mode. ECB mode is deterministic (the same plaintext block always encrypts to the same ciphertext block), making it vulnerable to replay attacks and pattern recognition.

### Is This an Actual Problem?

**Yes, this is a genuine security flaw.** Using the ECB mode compromises confidentiality because an attacker can observe patterns in the encrypted data, especially if the data has repetitive blocks. Furthermore, it lacks authentication, allowing potential tampering (integrity failures).

CodeQL appropriately recommends transitioning to authenticated, non-deterministic encryption modes like **GCM** (Galois/Counter Mode). A better instantiation would be

`Cipher.getInstance("AES/GCM/NoPadding")` , alongside a secure randomly generated Initialization Vector (IV) to prevent replay vulnerabilities.

## 4.2 Kingson's CodeQL Finding: Inefficient Regular Expression (ReDoS)

| Property | Value |
|---|---|
| **Rule** | `java/redos` |
| **Severity** | High |
| **File** | SplitAfterRadioButton.java |
| **Module** | `pdfsam-tools/pdfsam-simple-split` |

### What the Warning Says

CodeQL flags an inefficient regular expression used for validating input page numbers in `SplitAfterRadioButton.java`:

```
// SplitAfterRadioButton.java, Line 57
this.field.setValidator(Validators.regexMatching("^([1-9]\\d*(\\s*,\\s*)?)+$")); // ← CodeQL:
```

According to CodeQL (CWE-1333, CWE-400), this regular expression is vulnerable to **Regular Expression Denial of Service (ReDoS)**. The specific part `([1-9]\\d*(\\s*,\\s*)?)+` contains repetitions ( `+` and `*` ) that interact in a way that creates ambiguity. For strings starting with '1' and containing many repetitions of '1', the regular expression engine (which uses a backtracking non-deterministic finite automata) may take an exponential amount of time to evaluate, leading to performance degradation or a DoS attack.

### Is This an Actual Problem?

**Yes, this is a real vulnerability, though impact is limited in a desktop client.** A malicious actor could craft a specific, long input string (e.g., `"1111111111111111111111111111"` ) that forces the regex engine to backtrack exponentially, freezing the UI thread for an unacceptable amout of time while evaluating the input.

While the risk is lower because PDFsam is a desktop application (the user is only DOSing themselves), the performance issue is real. The fix is to rewrite the regular expression to remove the ambiguity between repetitions. A safer alternative would be to split the string by commas and then validate the individual components (e.g., `[1-9]\d*` ), or use a stricter regex like `^[1-9]\d*(?:\s*,\s*[1-9]\d*)*$` .

## 4.3 Zian's CodeQL Finding

| Property | Value |
|---|---|
| **Rule** | `java/useless-null-check` |
| **Severity** | Warning |
| **File** | TooltippedTableCell.java |
| **Module** | `pdfsam-ui-components` |

### What the Warning Says

CodeQL identifies a redundant and misleading null check on a field variable in `TooltippedTableCell.java`：

```
// TooltippedTableCell.java, Lines 42-49
@Override
public void updateItem(T item, boolean empty) {
    super.updateItem(item, empty);
    if (nonNull(item) && nonNull(tooltip)) {    // ← CodeQL: Useless null check
        setTooltip(tooltip);
    } else {
        setTooltip(null);
        }
    // ...
    }
```

According to CodeQL (CWE-561), the `nonNull(tooltip)` check is useless because `tooltip` is assigned a newly created object (e.g., `new Tooltip(...)` ). An object returned by the `new` keyword in Java can never be `null` , meaning this part of the conditional statement will always evaluate to `true` .

### Is This an Actual Problem?

**No, this is not a severe bug, but it is a code logic issue that indicates a misunderstanding.** A null check on a newly instantiated variable is superfluous and misleading to other developers reading the code, as it implies the reference could somehow be null, creating confusion about the lifecycle of the `tooltip` object.

While the program's behavior doesn't change, dead logic wastes cognitive space and slightly inflates compiled bytecode. The fix is to simply remove the `nonNull(tooltip)` condition from the `if` statement, streamlining the logic to just check `if (nonNull(item))` .

# 🔍 5. SpotBugs Detailed Findings — Per Team Member

## 5.1 Zhenyu's SpotBugs Finding: Inconsistent Synchronization

| Property | Value |
|---|---|
| **Bug Type** | `IS2_INCONSISTENT_SYNC` |
| **Category** | Multithreaded Correctness (MT_CORRECTNESS) |
| **Priority** | 2 (Medium) |
| **File** | ApplicationContext.java |
| **Module** | `pdfsam-core` |

### What the Warning Says

SpotBugs reports: "*Inconsistent synchronization of `org.pdfsam.core.context.ApplicationContext.runtimeState` ; locked 60% of time*"

This means the field `runtimeState` in `ApplicationContext` is accessed with synchronization (via `synchronized` blocks or methods) in some code paths but accessed without synchronization in others. SpotBugs detected that approximately 60% of accesses are synchronized, but the remaining 40% are not.

### Is This an Actual Problem?

**Yes, this is a genuine thread-safety concern.** If `runtimeState` is shared across threads (which it is, as `ApplicationContext` is a singleton), inconsistent synchronization can lead to:

- **Data races**: One thread may read a partially-updated reference
- **Visibility issues**: Changes made by one thread may not be visible to another due to JMM (Java Memory Model) caching

The fix would be to either synchronize *all* accesses to `runtimeState` or declare it as `volatile` if only reference visibility is needed.

## 5.2 Kingson's SpotBugs Finding

| Property | Value |
|---|---|
| **Bug Type** | `SING_SINGLETON_HAS_NONPRIVATE_CONSTRUCTOR` |
| **Category** | Correctness (CORRECTNESS) |
| **Priority** | 2 (Medium) |
| **File** | ApplicationContext.java |
| **Module** | `pdfsam-core` |

### What the Warning Says

SpotBugs reports: "*Class (org.pdfsam.core.context.ApplicationContext) using singleton design pattern has non-private constructor.*"

`ApplicationContext` follows the singleton pattern (only one instance should exist), but its constructor is not declared `private`. This means other code could potentially create additional instances, breaking the singleton guarantee:

```
public class ApplicationContext {
    // Constructor is package-private, not private
    ApplicationContext() {  // ← Should be private for a true singleton
        // ...
    }
}
```

### Is This an Actual Problem?

**Partially.** The constructor is package-private (default access), not `public`, so only classes in the same package can create new instances. In practice, PDFsam uses Jakarta CDI for dependency injection, which manages the singleton lifecycle at the framework level. However, leaving the constructor non-private is still a violation of the singleton pattern: any future class added to the `org.pdfsam.core.context` package could accidentally instantiate a second `ApplicationContext`. Making the constructor `private` and relying solely on CDI would be the safer approach.

## 5.3 Zian's SpotBugs Finding

| Property | Value |
|---|---|
| **Bug Type** | `DM_DEFAULT_ENCODING` |
| **Category** | Internationalization (I18N) |
| **Priority** | 1 (High — SpotBugs' highest severity) |
| **File** | ApplicationContext.java |
| **Module** | `pdfsam-core` |

### What the Warning Says

SpotBugs reports: "*Found reliance on default encoding in `org.pdfsam.core.context.ApplicationContext.lambda$registerScene$7(String) : String.getBytes()`* "

Somewhere in `ApplicationContext.registerScene()` , the code calls `String.getBytes()` without specifying a character encoding. This uses the platform's default encoding, which varies across operating systems:

- Windows: `windows-1252`
- Linux: `UTF-8`
- macOS: `UTF-8`

### Is This an Actual Problem?

**Yes, this is a real portability issue.** If the same PDFsam binary is run on different operating systems, `String.getBytes()` may produce different byte arrays for the same input string. This can cause:

- Corrupted data when the bytes are later decoded with a different encoding
- Different hash values on different platforms
- Subtle bugs that only manifest on specific OS configurations

The fix is straightforward: replace `str.getBytes()` with `str.getBytes(StandardCharsets.UTF_8)` to ensure consistent encoding regardless of platform.

# ⚖️ 6. Tool Comparison

## 6.1 Summary of Findings

| Aspect | CodeQL | SpotBugs |
|---|---|---|
| **Total Findings** | 84 | 47 |
| **Analysis Level** | Source code (AST + data flow) | Compiled bytecode |
| **Highest Severity** | High | High |
| **Primary Focus** | Security + Code Quality | Bug patterns + Code smells |

## 6.2 Overlap and Differences

| Dimension | CodeQL | SpotBugs |
|---|---|---|
| **Internal State Exposure** | ❌ Does not flag `EI_EXPOSE_REP` | ✅ Dominant finding (42 of 47 bugs) |
| **Cryptographic Security** | ✅ Flags weak algorithms (e.g., AES/ECB) | ❌ Limited cryptographic analysis |
| **Performance (ReDoS)** | ✅ Flags inefficient regex patterns | ❌ Does not analyze regex performance |
| **Thread Safety** | ❌ Limited MT analysis | ✅ Detects inconsistent synchronization |
| **Redundant Logic** | ✅ Detects useless null checks | ❌ Focuses primarily on bytecode patterns |
| **Default Encoding** | ❌ Does not flag `String.getBytes()` | ✅ Highest-priority finding (P1) |

## 6.3 Overlapping Findings

Both tools analyze the same codebase but approach it from fundamentally different angles. We found **minimal direct overlap** between findings:

- CodeQL found issues that SpotBugs missed entirely (weak cryptography, ReDoS vulnerabilities, redundant null checks)

- SpotBugs found issues that CodeQL missed entirely (internal state exposure, inconsistent synchronization, default encoding)
- **Zero identical findings** were reported by both tools for the same line of code

This is because the tools operate at different levels:

- **CodeQL** builds a semantic database from source code, tracing variables across methods and evaluating cryptographic and regex signatures, making it excellent at detecting *security-oriented* and *semantic logic* issues
- **SpotBugs** analyzes bytecode patterns, making it excellent at detecting *implementation-level* issues like missing defensive copies, improper synchronization, and encoding problems

## 6.4 Strengths and Weaknesses

| Tool | Strengths | Weaknesses |
|---|---|---|
| **CodeQL** | Deep semantic analysis; excellent security coverage; customizable query language (QL); integrates with GitHub UI for PR reviews | Higher setup complexity; requires full build to create database; can be slow on large projects; limited bytecode-level analysis |
| **SpotBugs** | Fast analysis; wide bug pattern library (400+); no build system integration needed beyond compiling; catches low-level Java issues (encoding, synchronization) | High false-positive rate for `EI_EXPOSE_REP` (many are by design); bytecode-only means it misses source-level patterns; no QL-like customization; limited security analysis |

## 6.5 Complementary Value

The two tools provide **complementary, non-overlapping** value:

```
      CodeQL                    SpotBugs
┌──────────────┐            ┌──────────────┐
│  Cryptography│            │  Thread      │
│  ReDoS (Perf)│            │  Safety      │
│  Data Flow   │            │  Encoding    │
│  Semantic    │            │  Internal    │
│  Logic       │    ∅       │  State       │
│  Redundant   │ (overlap)  │  Exposure    │
│  Checks      │            │  Singleton   │
│              │            │  Patterns    │
└──────────────┘            └──────────────┘
```

**Recommendation**: Use *both* tools in CI:

- **CodeQL** via GitHub Actions for security-focused scanning and PR-level feedback
- **SpotBugs** via Maven plugin for developer-focused bug detection during local builds

Together, they provide defense-in-depth against a wide spectrum of code quality and security issues that neither tool could catch alone.

# 📋 7. Summary

## 7.1 Files Changed

| File | Owner | Action | Description |
|---|---|---|---|
| .github/workflows/codeql.yml | Team | **NEW** | CodeQL analysis workflow for GitHub Code Scanning |

## 7.2 Commands Used

```
# Run SpotBugs on all non-GUI modules
mvn compile com.github.spotbugs:spotbugs-maven-plugin:4.8.6.6:spotbugs -pl pdfsam-model,pdfsa

# View SpotBugs XML reports
cat pdfsam-model/target/spotbugsXml.xml
cat pdfsam-core/target/spotbugsXml.xml
cat pdfsam-persistence/target/spotbugsXml.xml
cat pdfsam-service/target/spotbugsXml.xml

# Extract human-readable bug summary from SpotBugs XML
python3 -c "
import xml.etree.ElementTree as ET
for module in ['pdfsam-model','pdfsam-core','pdfsam-persistence','pdfsam-service']:
    tree = ET.parse(f'{module}/target/spotbugsXml.xml')
    bugs = tree.getroot().findall('.//BugInstance')
    print(f'{module}: {len(bugs)} bugs')
    for bug in bugs:
        print(f'  [{bug.get(\"priority\")}] {bug.get(\"type\")} - {bug.find(\".//LongMessage\
"

# Enable CodeQL (push the workflow to GitHub)
git add .github/workflows/codeql.yml
git commit -m "Add CodeQL analysis workflow"
git push origin master

# View CodeQL results on GitHub
# Navigate to: https://github.com/eric-song-dev/pdfsam/security/code-scanning
```

## 7.3 Key Takeaways

| Takeaway | Details |
| --- | --- |
| **Static analysis catches bugs that testing misses** | Thread safety issues ( `IS2_INCONSISTENT_SYNC` ) and encoding portability ( `DM_DEFAULT_ENCODING` ) are nearly impossible to find with unit tests alone |
| **Different tools find different things** | CodeQL and SpotBugs had **zero overlap** in findings — they are truly complementary |
| **Not all warnings are bugs** | Many SpotBugs `EI_EXPOSE_REP` findings are intentional design choices (e.g., exposing `ObservableValue` for JavaFX binding) |
| **Tool selection matters** | CodeQL excels at security/data-flow analysis; SpotBugs excels at implementation-level patterns |
| **CI integration is essential** | Running static analyzers on every push ensures new bugs are caught before they reach production |

# 🎯 8. Conclusion

This report demonstrates the application of **two complementary static analysis tools** — **CodeQL** and **SpotBugs** — to the PDFsam Basic project:

1. **CodeQL** (GitHub Code Scanning) found **84 findings** focused on cryptographic security, regular expression performance, semantic logic flaws, and general code quality
2. **SpotBugs** (bytecode analyzer) found **47 findings** focused on internal state exposure, thread safety, encoding portability, and singleton correctness
3. The tools had **zero overlapping findings**, confirming that they provide fundamentally different perspectives on the same codebase

Each team member analyzed one finding per tool:

| Member | CodeQL Finding | SpotBugs Finding |
|---|---|---|
| **Zhenyu Song** | Weak cryptographic algorithm `AES/ECB/PKCS5Padding` in `EncryptionUtils` | Inconsistent synchronization of `runtimeState` in `ApplicationContext` |
| **Kingson Zhang** | Inefficient regular expression (ReDoS) in `SplitAfterRadioButton` | Singleton with non-private constructor in `ApplicationContext` |
| **Zian Xu** | Useless null check in `TooltippedTableCell` | Reliance on default encoding `String.getBytes()` in `ApplicationContext` |

Static analysis complements the testing efforts from other parts by catching an entirely different class of defects — those that are difficult or impossible to find through execution-based testing alone.